Time Series Project

Sebastian Parasca

1 - Introduction

In this project, we will analyze the monthly sales performance in the automotive industry from January 1960 to December 1968. This is an interesting period in the automotive history which is characterized by various events, including: post-war economic growth in both US and Western Europe, the rising of middle-class purchasing power, where a car was not perceived as a luxury item anymore and the subsequent increase in consumer demand for vehicles.

2 - Analysis

We start the analysis by uploading the data, observing its structure, and then plotting the relevant graphs which are going to be useful in our analysis.

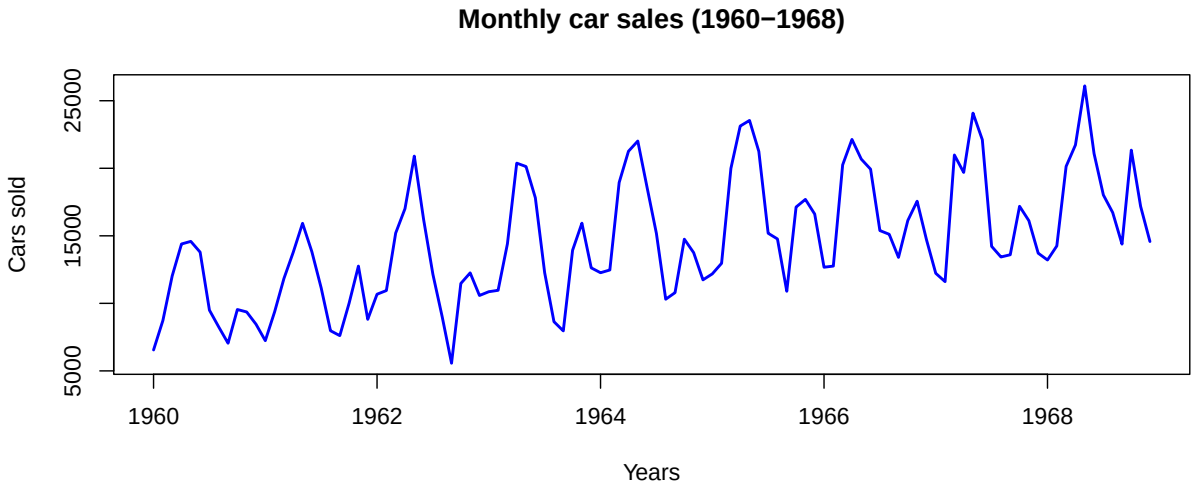
```
data <- read.csv(file.choose(), header = TRUE, sep = ",")
any(is.na(data))
```

```
[1] FALSE
```

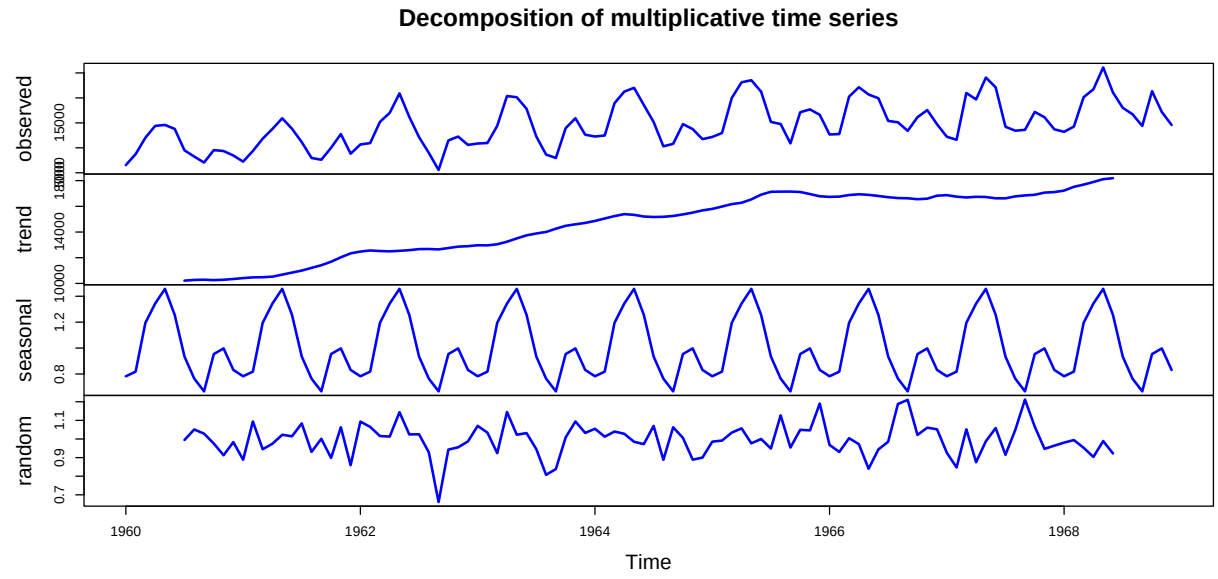
```
monthly_sales <- data$Sales
head(data, n = 5)
```

```
    Month Sales
1 1960-01  6550
2 1960-02  8728
3 1960-03 12026
4 1960-04 14395
5 1960-05 14587
```

```r
plot(ts(monthly_sales, frequency = 12, start = c(1960,1)),
     col = "blue", lwd = 2,
     ylab = "Cars sold",
     xlab = "Years",
     main = "Monthly car sales (1960-1968)")
```




```r
plot(decompose(ts(monthly_sales, frequency = 12, start = c(1960,1)),
     type = "multiplicative"),
     col = "blue", lwd = 2)
```



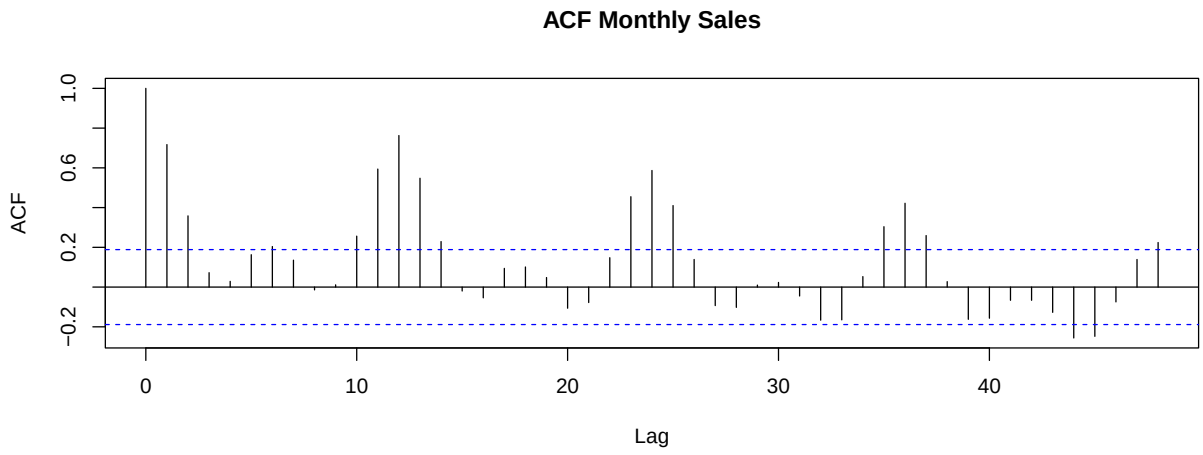
Observations:

The first thing we can notice from these plots is that the series shows a clear upward trend thorought the entire period, also confirmed by the trend plot in the decomposition graph, indicating that the mean is not constant over time. As the series progresses, the size of the fluctuations seems to increase which suggests that the variance is also time dependent and also justifies the choice for the multiplicative decomposition given the nature of the fluctuations. Furthermore, we can also see that the residuals in the decomposition graph are centered around 1 but the variance is still not constant which implies heteroscedasticity, for which we will later apply log transformations, in order to stabilize it.

Lastly, we can also observe a regular cyclical behaviour, which provides clear evidence of the fact that seasonality is present in the data, therefore after all the observations we can affirm that the series is clearly not stationary.

To get a better understanding of the seasonality aspect we can plot the ACF function.

```
acf(monthly_sales, lag.max = 48, main = "ACF Monthly Sales")
```

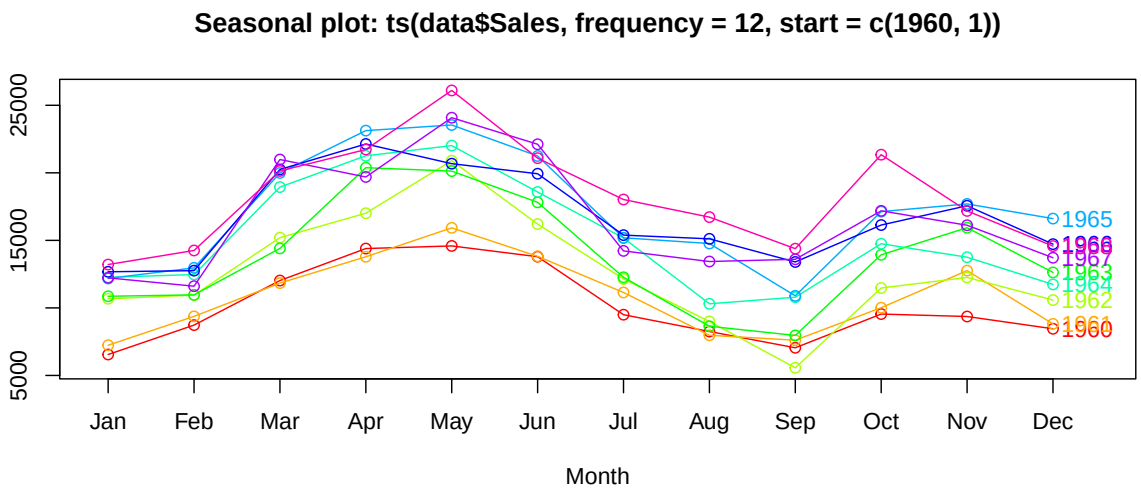


Observations:

The plot of the autocorrelation function provides strong evidence of seasonality in the data. In fact, as the ACF declines gradually, there are spikes outside the significance bands at lags 12, 24 and 36; corresponding to 1,2 and 3 year intervals respectively. This indicates that the car sale in any given month are strongly correlated with sales in the same month of the previous year, confirming our initial hypothesis.

We can go one step further and visualize the monthly car sales through a seasonal plot to have a better understanding of the seasonal component.

```
library(forecast)
seasonplot(ts(data$Sales, frequency = 12, start = c(1960,1)),
           year.labels = TRUE,
           col = rainbow(9))
```



From the seasonal plot we can see a recurring pattern across all years. Independetly of the year, the car sales have the same behaviour each month. They rise from January to March, reaching a peak in May, followed by a declining through the summer months until they partially recover towards October and Novembre. This confirms again the yearly seasonality aspect in the data.

3 - Transformations

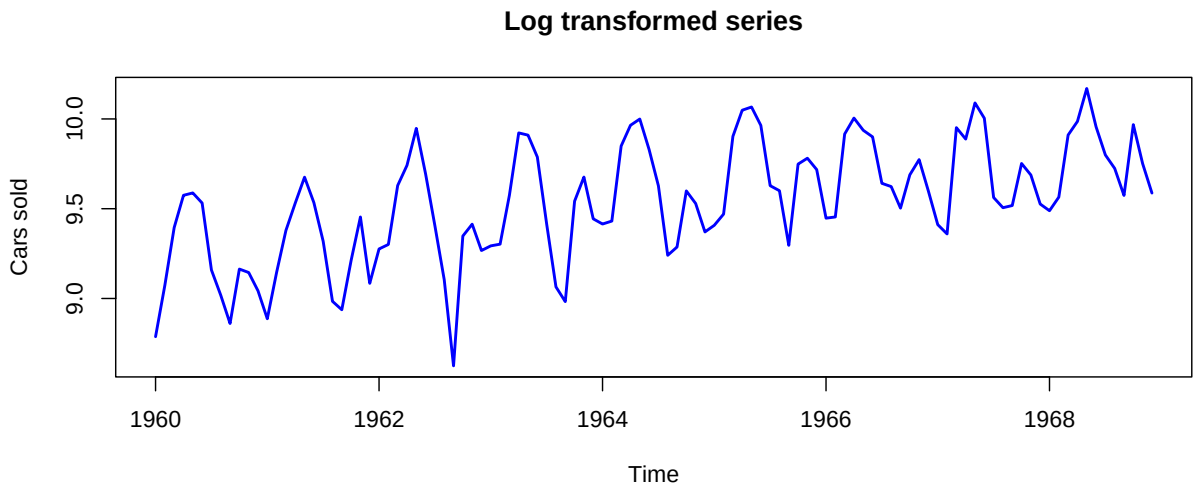
After the initial analysis, we will now perform various transformations with the goal of making the series stationary.

The main transformations are going to be:

1. Log Transformation (to stabilize the non-constant variance)
2. First Order Differencing (to remove the trend component and stabilize the mean)
3. Seasonal Differencing (to remove the seasonal component)

3.1 - Log Transformation

```
log_data <- log(data$Sales)
plot(ts(log_data, frequency = 12, start = c(1960,1)),
     col = "blue", lwd = 2,
     main = "Log transformed series",
     ylab = "Cars sold")
```

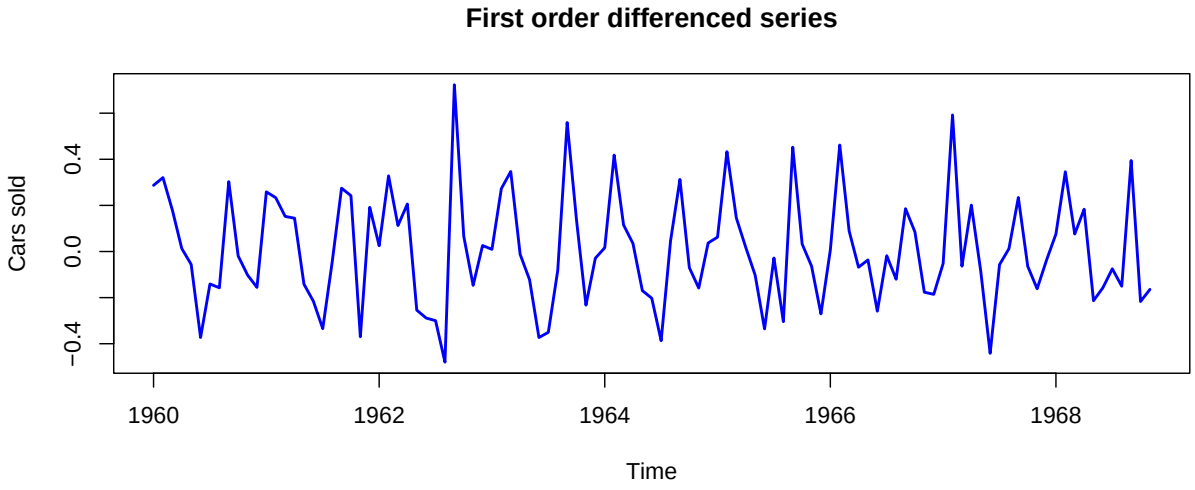


Observations:

After performing the log transformation we can notice that the fluctuations look a lot more uniform, especially the swings from 1964 - 1968 are not as large as they were in the original graph. Moreover, we can still see a clear trend in the data and the seasonal aspect is still present, therefore we can go on and apply the other transformations.

3.2 - First Order Differenceing

```
diff_data <- diff(log_data)
plot(ts(diff_data, frequency = 12, start = c(1960,1)),
     col = "blue", lwd = 2,
     main = "First order differenced series",
     ylab = "Cars sold")
```

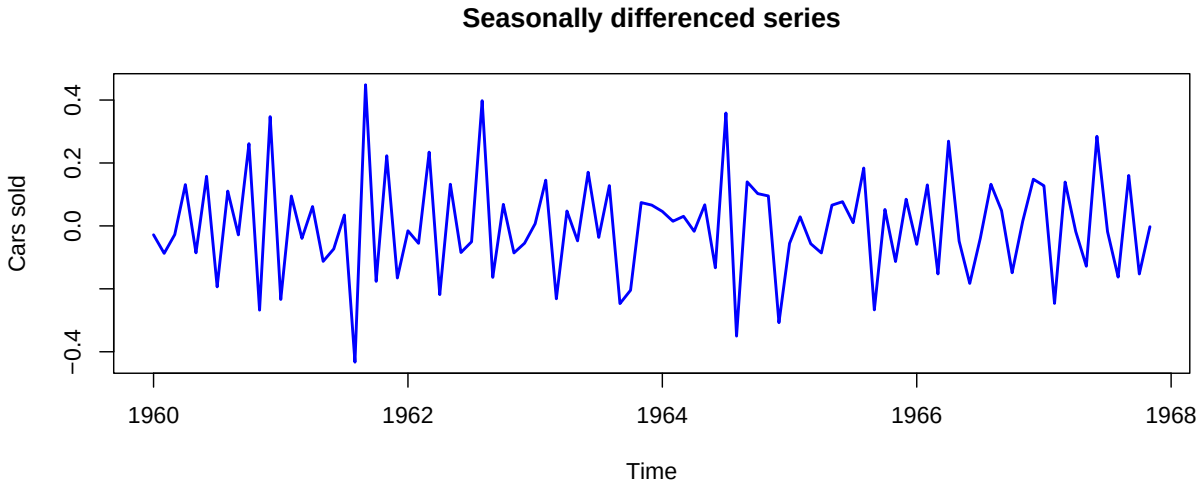


Observations:

Following the first order difference we can see that the trend component has been removed, the series oscillates between -0.4 and ~ 0.5 with no visible drift, but the seasonal component is still clearly visible, therefore we will perform a last transformation.

3.3 Seasonal Differencing

```
diff_seasonal <- diff(diff_data, lag = 12)
plot(ts(diff_seasonal, frequency = 12, start = c(1960,1)),
     col = "blue", lwd = 2,
     main = "Seasonally differenced series",
     ylab = "Cars sold")
```



Observations:

After the seasonal differencing performed with 12 lags we can see that the seasonal component has been removed, the series fluctuates around 0 and the variance is not perfectly constant but more stable than before, with a range between -0.4 and 0.4

The last test we will perform to ensure that the series is stationary after the transformations is the ADF test:

```
library(tseries)
adf.test(diff_seasonal)
```

```
    Augmented Dickey-Fuller Test

data:  diff_seasonal
Dickey-Fuller = -6.4391, Lag order = 4, p-value = 0.01
alternative hypothesis: stationary
```

Observations:

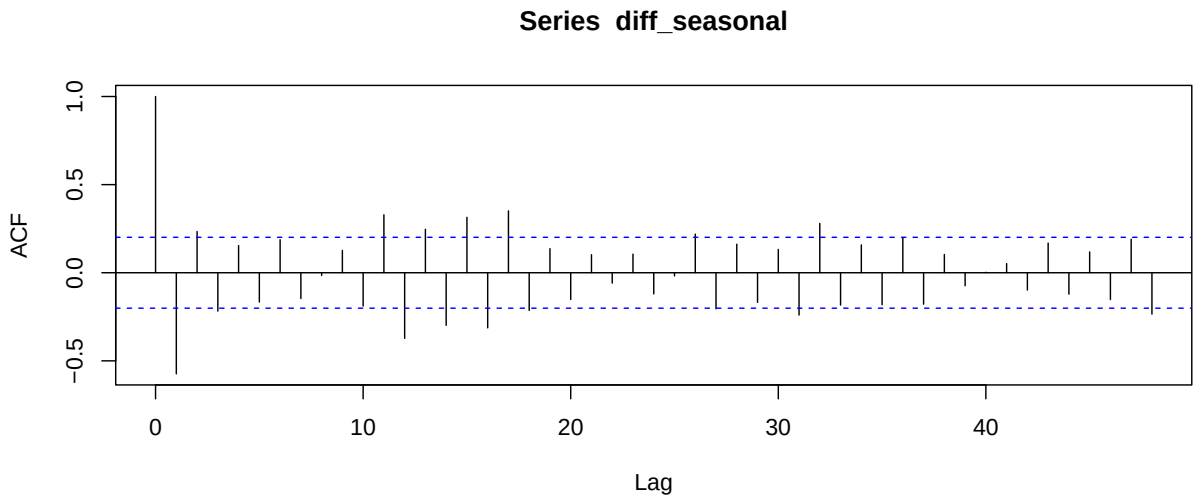
As the p-value is < 0.05, we reject the null hypothesis of the presence of a unit root. Therefore, we can conclude that the series is stationary after all the transformations performed.

4 - Modelling

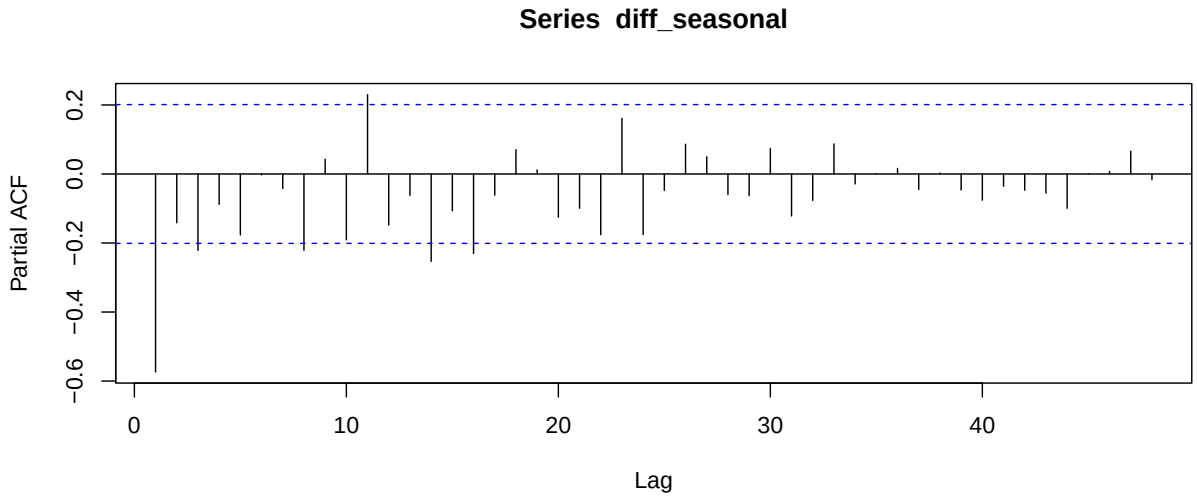
After the data has been analyzed and the various transformations have been applied, we can now try to find a model to forecast future sales.

Before finding a model, we should make some considerations by looking at the ACF and PACF.

```
acf(diff_seasonal, lag.max = 48)
```




```
pacf(diff_seasonal, lag.max = 48)
```



Observations:

From the ACF plot we can notice that only the first lag shows a clear significant spike while the remaining ones are within the significant bounds or borderline, suggesting that we should consider a MA(1).

Meanwhile, from the PACF plot we see quite the same behaviour, in fact the PACF cuts off after lag one, therefore we should consider an AR(1).

Since both ACF and PACF cut off after lag one, this strongly points to an ARMA(1,1) but given that the series has a trend component and seasonality is present, we should consider an ARIMA(1,1,1), where the integrating part takes into account the trend and we should also specify a period of 12 months for the seasonality component.

Furthermore, given that the model performance is measured with the AIC, which strongly penalizes the addition of more parameters, we would also consider an ARIMA(0,1,1) and an ARIMA(1,1,0) to see if the additional complexity is justified.

In order to choose the parameters for the seasonal component (P,D,Q) we would look at the seasonal lags 12, 24 and 36 in the PACF and ACF. First of all, we will choose D = 1 due to the yearly seasonality in the data which needs differencing at lag 12, moreover after checking for a seasonal AR (PACF) we notice that at the representative lags there is no significant spike. At last, after looking at the ACF for a seasonal MA we notice a significant spike at lag 12, therefore Q = 1.

We would train the model on 7 years of data (1960-1967) and compare it to the last year (1968).

```
log_sale_ts <- ts(log(data$Sales), frequency = 12, start = c(1960,1))

train <- window(log_sale_ts, end = c(1967,12))
test <- window(log_sale_ts, start = c(1968,1))
```

```
model1 <- arima(train, order = c(0,1,1),
                seasonal = list(order = c(0,1,1), period = 12))

model2 <- arima(train, order = c(1,1,0),
                seasonal = list(order = c(0,1,1), period = 12))

model3 <- arima(train, order = c(1,1,1),
```

```
                seasonal = list(order = c(0,1,1), period = 12))

aic_values <- c("ARIMA(0,1,1)" = AIC(model1),
  "ARIMA(1,1,0)" = AIC(model2),
  "ARIMA(1,1,1)" = AIC(model3))
print(aic_values)
```

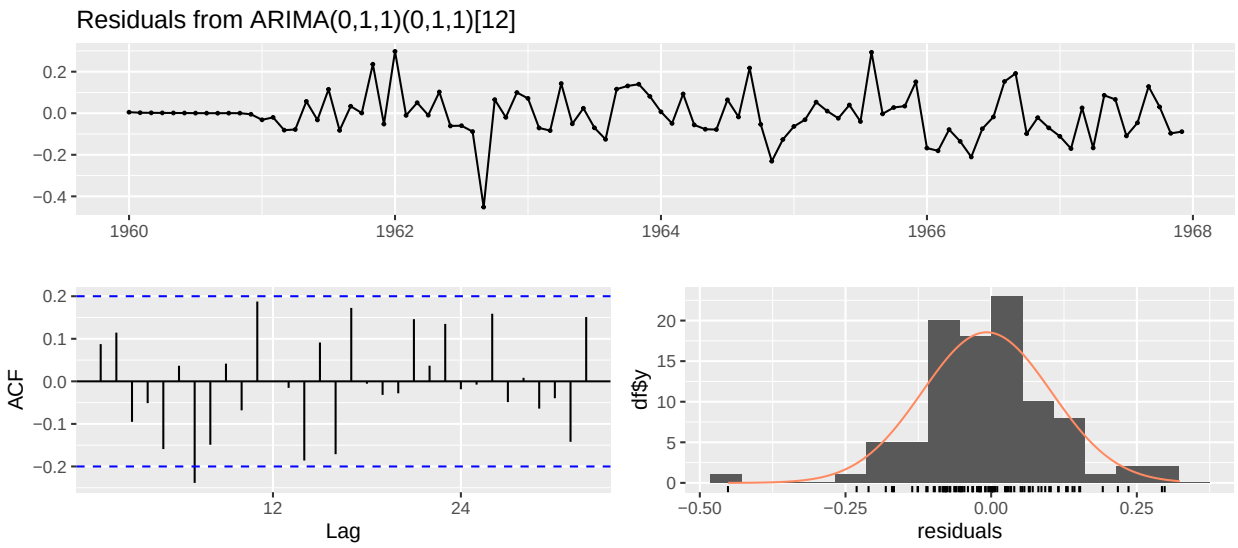
```
ARIMA(0,1,1) ARIMA(1,1,0) ARIMA(1,1,1)
  -108.30411    -99.65616   -108.02990
```

Observations:

After fitting the models, we can see, that based on AIC the simpler model, MA(1) is slightly better than an ARIMA(1,1,1) therefore we would choose the ARIMA(0,1,1) as it performs better without the extra AR component.

In order to see if the latter model does a good job in capturing all the structure in the data, we will now analyze the distribution of the errors/residuals:

```
checkresiduals(model1)
```




```
    Ljung-Box test
```

```
data:  Residuals from ARIMA(0,1,1)(0,1,1)[12]
Q* = 31.071, df = 17, p-value = 0.01958

Model df: 2.   Total lags used: 19
```

Observations:

After performing the Ljung-Box test we can see that the null hypothesis of the residuals being white noise is rejected (p-value < 0.05), therefore there is still some autocorrelation structure in the residuals that our model did not capture.

If we take a closer look at the other plots, we can notice that the residuals are roughly centered around 0 without any visible pattern, but there is a huge spike roughly at the end of 1962 caused by a sharp decline in sales, which the model did not capture. The rejection of the null hypothesis is likely due to this outlier which also causes the distribution of the residuals to be left-skewed.

As we cannot provide a clear explanation of what happened during that year we will keep this outlier in the data for now.

4.1 - Forecasting

After training the model, we will now use it to forecast the last year car sales (1968) and we will assess model performance through the MAE as the RMSE is sensitive to outliers given the squaring factor (the errors get amplified).

```r
library(forecast)
fc_model <- forecast(model1, h = 12)
acc <- accuracy(fc_model, test)
df_acc <- as.data.frame(acc)
df_acc$RMSE <- (exp(df_acc$RMSE)-1)* 100
df_acc$MAE  <- (exp(df_acc$MAE)-1)* 100

df_acc_partial <- df_acc[, c("RMSE", "MAE", "MASE")]
df_acc_partial
```

```
                    RMSE      MAE      MASE
```

```
Training set 11.64803 8.274667 0.6948833
Test set     12.62666 9.662897 0.8062370
```

Observation:

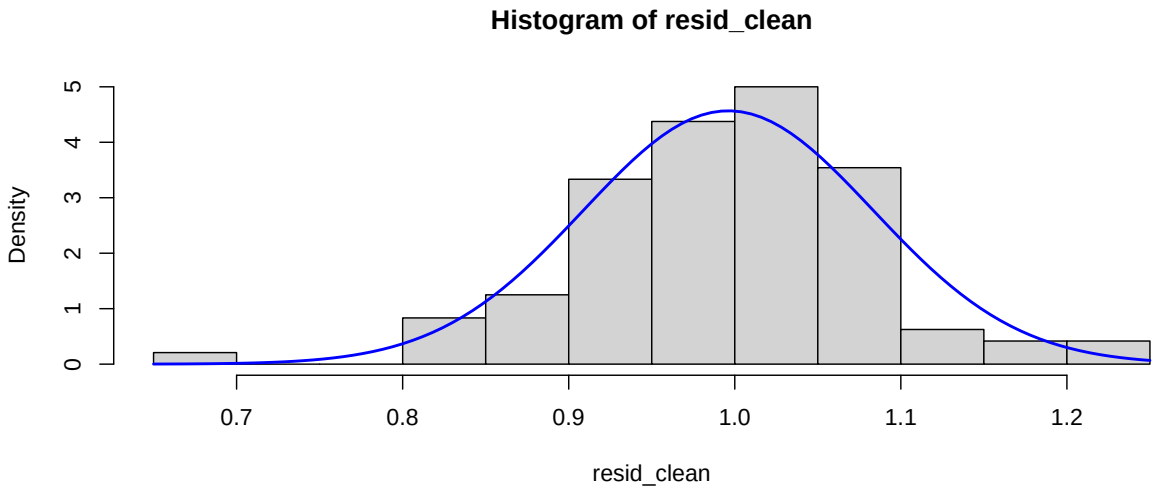
After forecasting, we can observe that on average our predictions are off by 9.66% (MAE), which is quite reasonable, and the RMSE is indeed higher as we previously stated, due to the presence of the 1962 outlier. At last we can observe a MASE of 0.80 which suggests that our model is ~ 20% better than a naive model. As the residuals we previously analyzed are well-behaved in terms of white-noise, the most plausible explanation for the MASE not being lower is that the outlier distorts the model perfomance given its presence in the training data.

4.2 - Outlier Handling

Given that the outlier was flagged in the multiplicative decomposition, we will perform a visual inspection of its residuals.

```r
ts_data <- ts(data$Sales, start = c(1960, 1), frequency = 12)
decomp_data <- decompose(ts_data, type = "multiplicative")
residuals_data <- decomp_data$random
resid_clean <- na.omit(residuals_data)

hist(resid_clean, prob = TRUE)
curve(dnorm(x, mean = mean(resid_clean), sd = sd(resid_clean)),
      add = TRUE, col = "blue", lwd = "2")
```




```
std_residuals <- (resid_clean - mean(resid_clean)) / sd(resid_clean)

which(abs(std_residuals) > 2.5)
```

```
[1] 27
```

```
cycle(resid_clean)[27]
```

```
[1] 9
```

Observations:

After plotting the residuals of the multiplicative decomposition, we can see an observation at ~0.7 which is greater than 2.5 standard deviations from the mean, making it and outlier in the data. This corresponds to car sales in September 1962, which amounts to $5568.

Given the nature of the data (seasonality), sales tend to have a decrease during the Fall period, especially in September. Therefore, the most suitable operation is to change the value of the outlier with the average car sales in the month of September across all years, such that the underlying structure remains intact.

```
sept_mean <- mean(data$Sales[cycle(ts_data) == 9 & data$Sales != 5568])

data$Sales_clean <- data$Sales
data$Sales_clean[cycle(ts_data) == 9 & floor(time(ts_data)) == 1962] <- sept_mean
```

4.3 - Re-fitting the model

After changing the value of the outlier with the mean, we will train again the 3 models previously presented and choose the best one by comparing the AIC.

```
log_sale_ts_clean <- ts(log(data$Sales_clean), frequency = 12,
                        start = c(1960,1))
train_clean <- window(log_sale_ts_clean, end = c(1967,12))
test_clean <- window(log_sale_ts_clean, start = c(1968,1))
```

```
model1_clean <- arima(train_clean, order = c(0,1,1),
                seasonal = list(order = c(0,1,1), period = 12))

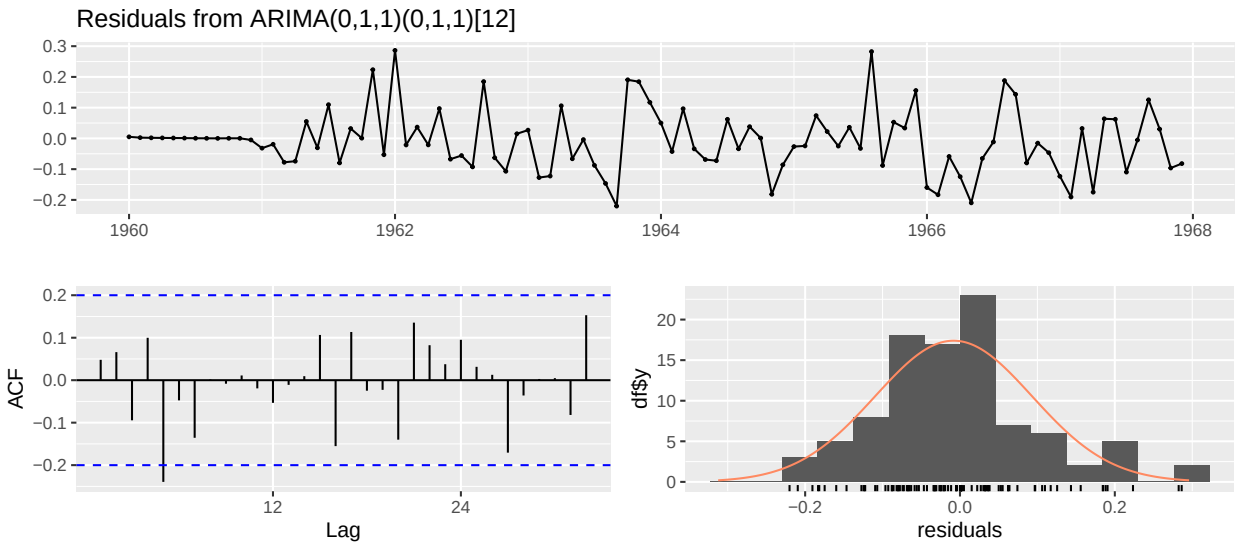
model2_clean <- arima(train_clean, order = c(1,1,0),
                seasonal = list(order = c(0,1,1), period = 12))

model3_clean <- arima(train_clean, order = c(1,1,1),
                seasonal = list(order = c(0,1,1), period = 12))

aic_values <- c("ARIMA(0,1,1)" = AIC(model1_clean),
  "ARIMA(1,1,0)" = AIC(model2_clean),
  "ARIMA(1,1,1)" = AIC(model3_clean))
print(aic_values)
```

```
ARIMA(0,1,1) ARIMA(1,1,0) ARIMA(1,1,1)
   -120.1706    -108.7619    -118.8598
```

```
checkresiduals(model1_clean)
```




```
    Ljung-Box test

data:  Residuals from ARIMA(0,1,1)(0,1,1)[12]
Q* = 16.917, df = 17, p-value = 0.46

Model df: 2.   Total lags used: 19
```

Observations:

First of all, we can already see an improvement in the Ljung-Box test, as the null hypothesis of the residuals being white noise is not rejected (p-value > 0.05). Furthermore, we can notice that the time series graph of the residuals is quite similiar to a white noise process, fluctuating around zero and with a constant variance, remarking the fact that our model captured the structure of the data.

Moreover, the best performing model is still the ARIMA(0,1,1) according to the AIC, which performs slightly better than an ARIMA(1,1,1) hence we will choose the former one.

4.3 Forecasting Again

Lastly, we will test the model again on the last year of observations and compute the various accuracy metrics to see if there was an improvement.

```r
library(forecast)
fc_model_clean <- forecast(model1_clean, h = 12)
acc_clean <- accuracy(fc_model_clean, test_clean)
df_acc_clean <- as.data.frame(acc_clean)
df_acc_clean$RMSE <- (exp(df_acc_clean$RMSE)-1)* 100
df_acc_clean$MAE  <- (exp(df_acc_clean$MAE)-1)* 100

df_acc_partial_clean <- df_acc_clean[, c("RMSE", "MAE", "MASE")]
df_acc_partial_clean
```

```
                RMSE      MAE      MASE
Training set 10.64745 7.853439 0.6629916
Test set     11.46855 8.569779 0.7210434
```

5 - Conclusion

We conclude the analysis by stating that the outlier handling had a positive effect. After training and testing, we can notice an improvement of the MASE from 0.8 to 0.72 which implies that our model performs better than a naive baseline model by ~28%. Moreover, the MAE decreased, reducing the average error in the prediction to roughly 8.57%. We should also notice that the training set MASE is 0.66 while the test set MASE is 0.72. This suggests the presence of mild overfitting during training. However, the model generalizes quite well to unseen data and an 0.06 increase in the MASE is acceptable. Further aspects should be considered in order to improve the model such as alternative forecasting models which could improve the accuracy.

Source Data: https://www.kaggle.com/datasets/rassiem/monthly-car-sales

ADF Test: https://www.geeksforgeeks.org/machine-learning/augmented-dickey-fuller-adf/

Ljung - Box Test: https://www.statology.org/ljung-box-test/

MAE-MASE-RMSE:

1) https://dev.to/mondal_sabbha/understanding-mae-mse-and-rmse-key-metrics-in-machine-learning-4la2

2) https://www.pmorgan.com.au/tutorials/mae,-mape,-mase-and-the-scaled-rmse/

Decompositions: https://otexts.com/fpp2/components.html